Sentiment Analysis of Cinematic Reviews: A Comprehensive Data Mining Approach

Zhi Liu
Dept. of Computer Science
ID:59363954

Siqi Sun
Dept. of Computer Science
ID:59414589

Zihan Lin
Dept. of Computer Science
ID:59801381

GenAI Acknowledgment

This document was edited with AI to improve grammar and readability. The ideas and content are the author's own.

Contents

1 Introduction **1**

2 System Modeling and Structure **1**

3 Dataset and Exploratory Analysis **2**

3.1 Datasets Description 2

3.2 Exploratory Analysis 3

4 Methodology **5**

4.1 Data Preprocessing 5

4.2 Feature Engineering 6

4.2.1 V1: TF-IDF Unigram 6

4.2.2 V2: TF-IDF Unigram + Bigram 7

4.2.3 V3: Statistical Features 7

4.3 Model 7

4.3.1 VADER Baseline 7

4.3.2 Logistic Regression 8

4.3.3 Linear Support Vector Classifier 8

4.3.4 Decision Tree 9

4.3.5 Random Forest 10

4.3.6 Model Selection 10

5 Experimental Setup **11**

6 Results and Discussion **11**

6.1 Overall Model Comparison 11

6.2 Error Analysis 12

6.3 Feature Interpretation 13

6.4 Influence of Review Length on Prediction 15

6.5 Summary of Analytical Findings 16

7 Conclusion and Future Work **16**

Abstract

This study presents a comprehensive data mining approach for sentiment analysis of IMDB movie reviews. We develop a robust end-to-end pipeline including rigorous text cleaning, feature engineering, model training, and evaluation. Three feature sets are designed: TF-IDF unigrams, TF-IDF unigrams with bigrams, and dense statistical features capturing textual and sentiment-specific patterns. Five models—VADER, Logistic Regression, Linear Support Vector Classifier (LinearSVC), Decision Tree, and Random Forest—are evaluated to compare rule-based, linear, and tree-based approaches. Results show that linear classifiers trained on TF-IDF features, particularly **LinearSVC**, achieve the highest performance (**accuracy 0.8953, F1-score 0.8961**), while tree-based models perform lower (accuracy $\approx$ 0.72–0.74), highlighting the importance of lexical and phrase-level patterns for sentiment recognition. Error analysis identifies negation scope, sarcasm, and topic drift in long reviews as primary challenges. Through systematic comparison of different models, we find that linear classifiers are most effective when leveraging TF-IDF features, especially bigrams, indicating that careful text cleaning and local context features significantly impact model performance. Future work may incorporate Transformer-based or other context-aware models to better capture sarcasm and complex sentiment shifts.

Keywords: Sentiment Analysis, IMDB Dataset, Data Mining, TF-IDF, Machine Learning

1 Introduction

Sentiment analysis, or opinion mining, is the computational study of people's opinions, sentiments, emotions, and attitudes expressed in written language. In the contemporary digital ecosystem, the proliferation of user-generated content on platforms such as review sites, social media, and forums has rendered sentiment analysis an indispensable tool within the broader field of data mining. Extracting actionable insights from this vast repository of subjective text enables organizations to gauge consumer satisfaction, monitor brand reputation, and inform strategic decision-making processes.

Despite its utility, sentiment analysis presents significant challenges. Textual data is inherently unstructured and frequently fraught with noise, including idiosyncratic formatting, grammatical inconsistencies, and context-dependent semantic modifiers such as negations. The motivation for this project stems from the necessity to construct a robust, end-to-end data mining pipeline capable of navigating these complexities.

The primary contribution of this project is the systematic deconstruction of the sentiment analysis workflow. By leveraging the large-scale IMDB movie review dataset, we conduct a rigorous exploratory data analysis to expose underlying data quality issues. Subsequently, we formalize a targeted preprocessing strategy and design multiple feature spaces—ranging from sparse vocabulary-based representations to dense statistical summaries. This structured approach not only facilitates a comparative analysis of different feature engineering techniques but also ensures methodological soundness through strict adherence to data leakage prevention principles.

2 System Modeling and Structure

Our system architectureshown in Figure 1, follows a three-stage pipeline that begins with **Data Processing**, where we perform the IMDB ingestion, text cleaning, feature engineering, and exploratory analysis to validate quality. The resulting standardized artifacts are passed to the **Modeling** stage, where multiple learners—VADER, Logistic Regression, Linear SVM, Decision Tree, and Random Forest—are trained and evaluated under consistent metrics.

The final stage, **Visualization & Reporting**, consumes the model outputs to produce plots, dashboards, and written reports, thereby transforming the experimental findings into actionable deliverables. External IMDB reviews act as the sole input to this pipeline, and the reporting layer delivers the final project outputs.

In the visualization and analysis stage, we conduct a comparative evaluation across all candidate models (VADER, Logistic Regression, Linear SVM, Decision Tree, Random Forest): metrics and predictions are consistently read from model outputs, performance comparison line/bar charts, confusion matrix heatmaps, and error distribution plots are generated, allowing a clear presentation of differences in accuracy, recall, F1 score, etc., while leveraging visualization to gain insights into misclassification patterns and model strengths/weaknesses.

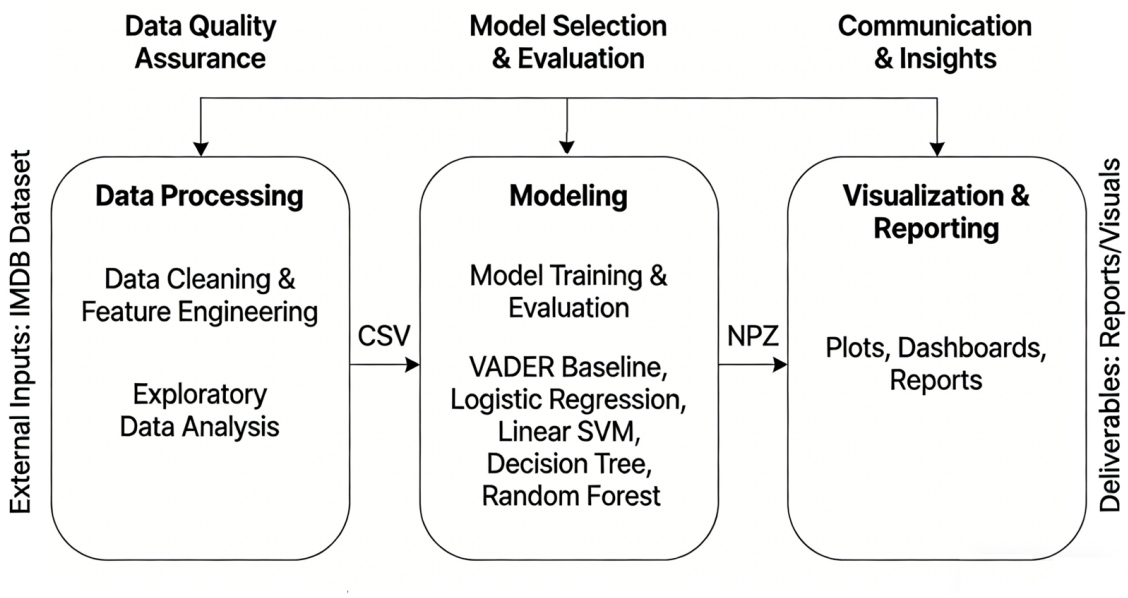

Figure 1: System Architecture

3 Dataset and Exploratory Analysis

3.1 Datasets Description

The analysis utilizes the widely recognized IMDB movie review dataset, a benchmark corpus for binary sentiment classification. The dataset comprises a total of 50,000 review samples, which have been partitioned equally into training and testing subsets, each containing 25,000 samples. A critical and highly beneficial characteristic of this dataset is its perfect class balance; both the training and testing sets consist of an exact 1.0 ratio of positive to negative reviews, with 25,000 instances belonging to each class overall. Furthermore, initial structural validations confirm that the dataset is highly complete, containing zero missing or empty reviews, providing a solid foundation for subsequent modeling.

Table 1: Dataset Description

Field Name	Data Type	Number of Samples	Description
ID	Integer	50,000	Unique identifier for each sample
Text	Text	50,000	Content of the review or text data
Label	Categorical	50,000	Positive/Negative sentiment label
Timestamp	DateTime	50,000	Collection time of the data
Source	Text	50,000	Data source or channel

The perfect symmetry depicted in Table 1 eliminates the need for complex class-balancing techniques, such as synthetic minority over-sampling, allowing the evaluation metrics to directly reflect the true performance of the predictive models.

3.2 Exploratory Analysis

Data Quality Analysis

While structurally complete, the dataset exhibits significant artifactual noise typical of web-scraped text. An inspection of the corpus reveals that 29,202 reviews contain HTML tags, with an overwhelming majority (29,200) explicitly containing line break tags. Additionally, the presence of URLs is noted in 202 reviews, alongside 4,660 instances of non-ASCII characters, which could introduce erratic tokens during vectorization. The dataset also suffers from excessive typographical noise, as evidenced by 12,662 reviews containing repeated punctuation sequences. Finally, a duplicate analysis identified 824 duplicate rows clustered into 406 distinct groups. Fortunately, no conflicting labels were found among these duplicates, but their presence necessitates deduplication to prevent model bias and artificial inflation of evaluation metrics.

Text Length Analysis

An analysis of text length reveals a high degree of variability and a pronounced right-skewed distribution. The average review contains approximately 231 words and 1309 characters. However, the median values are significantly lower, sitting at 173 words and 970 characters, respectively. The shortest review is exceptionally brief at merely 4 words, while the longest extends to 2,470 words.

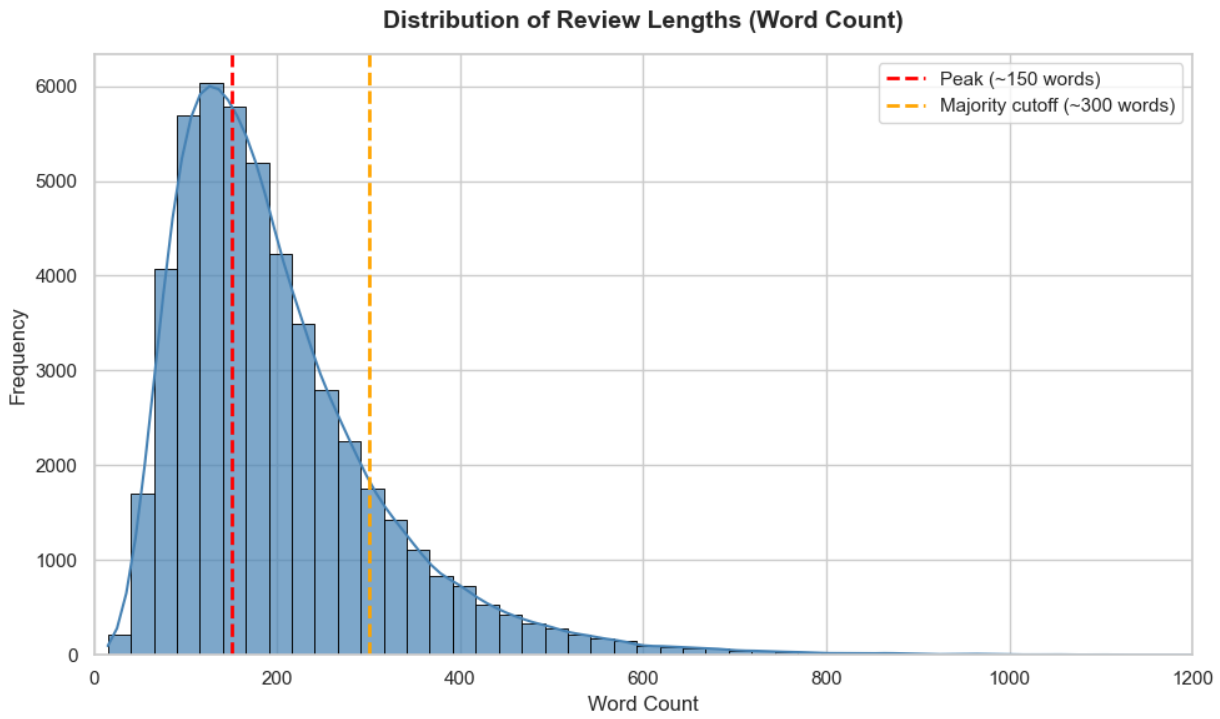

Figure 2: Distribution of review lengths (word count), illustrating a pronounced right tail with a majority of reviews falling under 300 words.

Figure 2 illustrates this long-tail phenomenon. The 95th percentile for word count is 590, indicating that while most users write concise reviews, a small fraction contributes highly verbose critiques.

Interestingly, the length distributions remain highly consistent across both the train/test splits and the positive/negative labels, suggesting that review length alone is not intrinsically indicative of sentiment or heavily biased across the partitions.

Vocabulary and Word Analysis

The corpus constitutes a massive vocabulary space, containing 11,753,379 total tokens mapped to 121,589 unique words. As expected in natural language, the frequency distribution is heavily dominated by English stopwords. The definite article ”the” is the most frequent token, appearing 666,832 times, followed closely by conjunctions and prepositions. Notably, the HTML line break artifact ”br” ranks as the 7th most frequent token overall (201,954 occurrences), further underscoring the urgent need for textual sanitization.

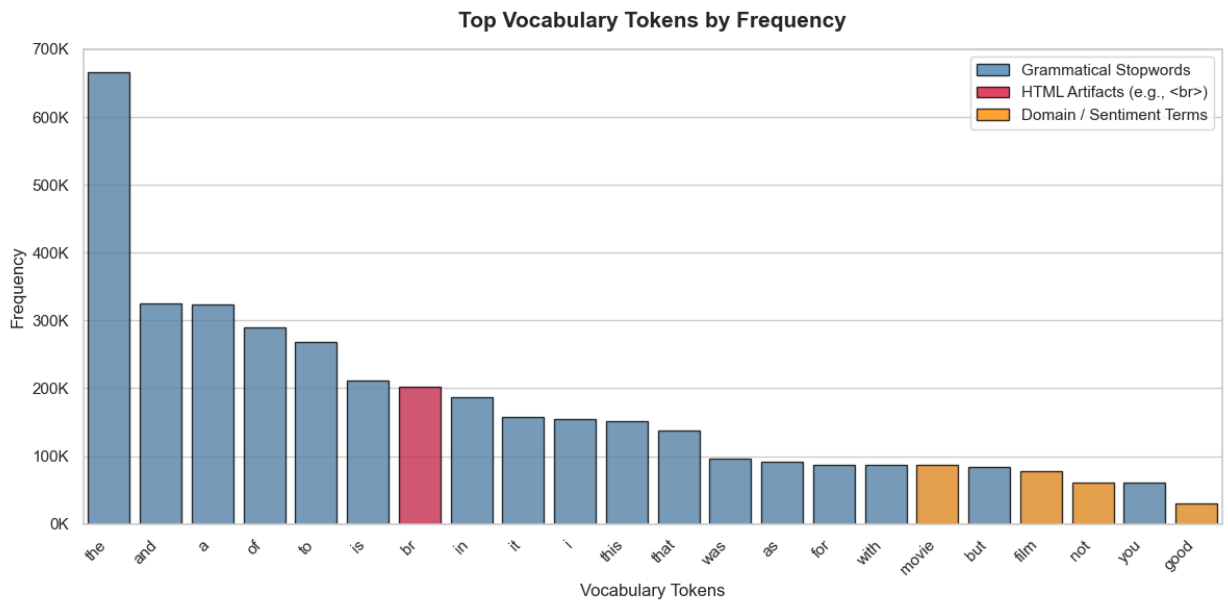

Figure 3: Frequency of top vocabulary tokens, highlighting the prevalence of stopwords, HTML artifacts, and domain terms.

Figure 3 contextualizes the vocabulary distribution. Beyond stopwords, explicit sentiment-bearing adjectives occur with high frequency. Positive descriptors such as ”good” (29,655 occurrences), ”great” (18,121), and ”best” (12,588) frequently contrast with negative counterparts like ”bad” (18,395) and ”worst” (5,326). Crucially, negations are profoundly prevalent. The word ”not” appears 60,720 times, alongside high frequencies for ”no” (25,262) and ”don’t” (16,850).

Observations

The exploratory analysis concretizes the requirements for the preprocessing pipeline. The models will be severely bottlenecked by the raw data unless HTML tags and non-ASCII characters are explicitly stripped. The sheer dominance of stopwords indicates that dimensionality reduction via stopword removal will be highly beneficial. However, the extraordinarily high frequency of negation terms strictly dictates that any stopword removal strategy must carefully preserve negating modifiers; removing words like ”not” would fundamentally invert the semantic truth of phrases like ”not good”,

severely crippling the model's predictive capacity.

4 Methodology

4.1 Data Preprocessing

Figure 4 shows the data cleaning steps applied prior to feature extraction. Raw text reviews are first stripped of HTML tags and converted to lowercase. Numeric tokens are removed while applying Porter stemming with careful handling of negations. Subsequent normalization collapses spaces, newlines, and standardizes punctuation. Finally, a split-aware deduplication step ensures that the cleaned reviews are ready for reliable feature engineering without introducing artificial bias.

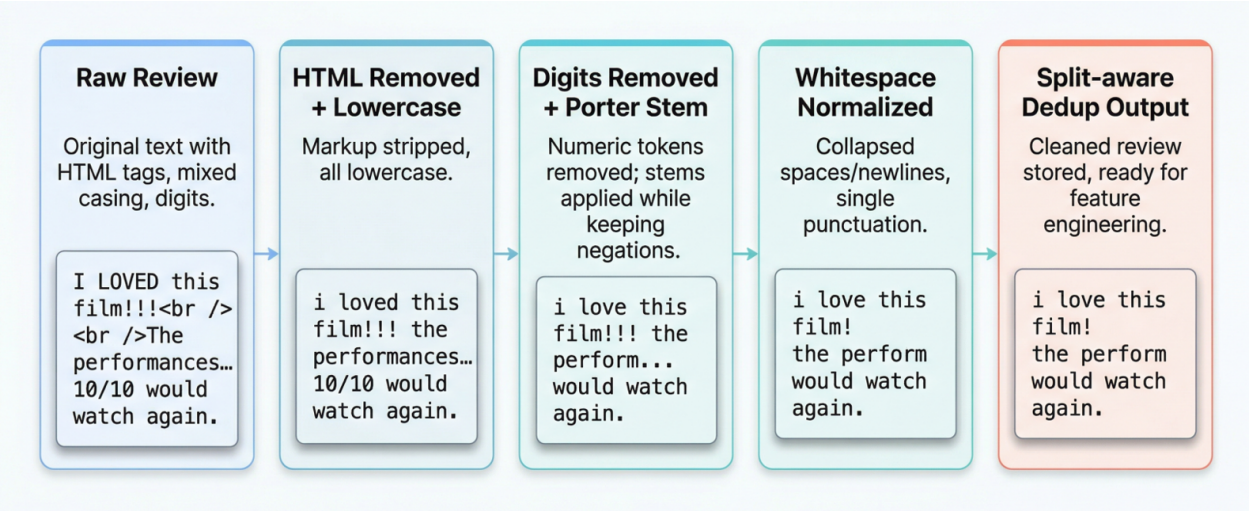

Figure 4: Data Cleaning Process for Text Preprocessing. The flow illustrates the transformation from raw reviews to cleaned and deduplicated text, including HTML removal, lowercasing, numeric token removal, stemming while preserving negations, whitespace normalization, and split-aware deduplication for feature engineering.

To transform the unstructured text into a machine-readable format, a sequential preprocessing pipeline is applied to the corpus. The first step involves rigorous HTML removal; given that nearly 60 percent of the dataset contains structural tags, stripping these artifacts prevents the vectorizer from treating markup as meaningful vocabulary. All text is subsequently converted to lowercase to ensure uniformity, preventing the algorithm from treating capitalized and uncapitalized versions of the same word as distinct features [3].

Following this, digits and non-alphabetic characters are removed, as numerical values in movie reviews generally offer negligible sentiment value compared to adjectives. Whitespace normalization is then applied to collapse multiple spaces and erroneous tabs into single spaces, ensuring clean tokenization. Duplicate handling is enforced by dropping the 824 identified redundant rows to prevent data leakage and overfitting [2].

Crucially, standard stopword removal is modified to preserve negations. While words like "the" and

”is” are discarded to reduce dimensional noise, terms such as ”not”, ”no”, ”didn’t”, and ”wasn’t” are explicitly retained. Finally, stemming is utilized to reduce words to their morphological roots (e.g., converting ”running”, ”runs”, and ”ran” to ”run”). This step significantly compresses the vocabulary space, aggregating the frequencies of semantically identical terms and mitigating the curse of dimensionality.

4.2 Feature Engineering

To adequately capture the textual information, three distinct feature sets are formulated, each serving a specific modeling paradigm. Figure 5 depicts the feature extraction pipeline applied to the cleaned review dataset. Three separate feature sets are generated: V1 employs TF-IDF unigrams fitted on the training set; V2 extends this to unigrams and bigrams, sharing a vectorizer across train and test matrices; V3 computes statistical features such as character and word counts, punctuation sequences, negation frequency, and lexicon-based positive/negative scores. Each pipeline branch produces corresponding artifact files including `.npz` matrices, vectorizer objects, and JSON metadata, which are used in subsequent model training stages.

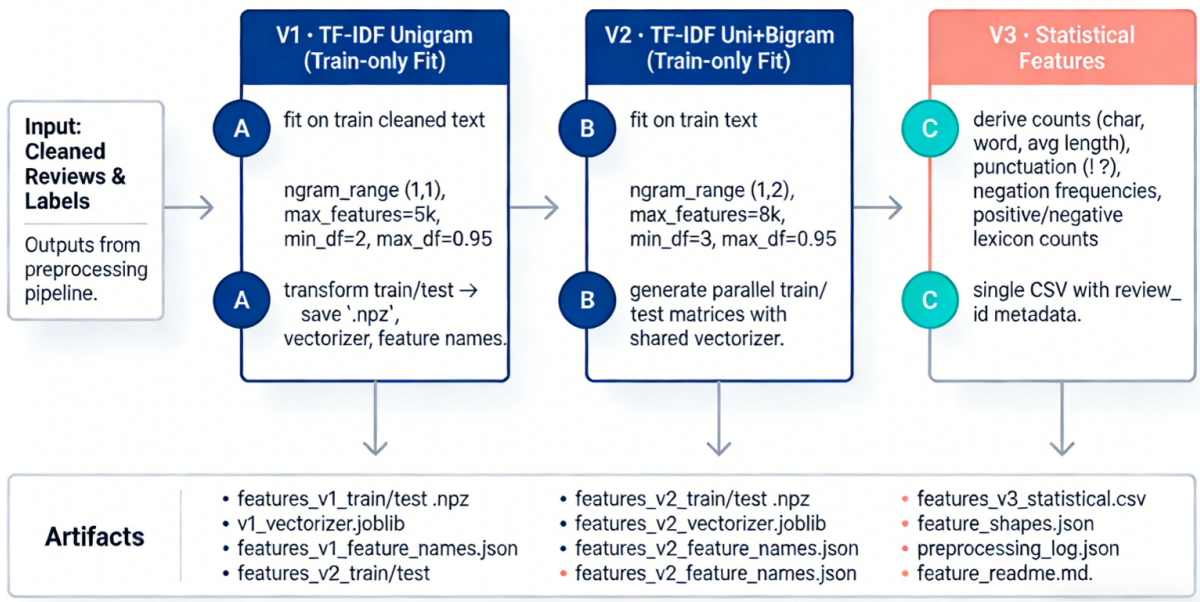

Figure 5: Feature Extraction Pipeline for Text Data. The diagram illustrates three feature sets: V1 (TF-IDF Unigram), V2 (TF-IDF Uni+Bigram), and V3 (Statistical Features). Each branch shows preprocessing steps, vectorization or feature derivation, and output artifacts generated for model training and analysis.

4.2.1 V1: TF-IDF Unigram

The first version, V1, utilizes Term Frequency-Inverse Document Frequency based solely on unigrams (single words) [4, 5]. This technique scales the raw frequency of a word within a specific

review by the inverse of its frequency across the entire corpus, effectively penalizing ubiquitous words and highlighting terms that are uniquely significant to individual documents. V1 serves as the foundational baseline for this project. Because it generates a highly sparse but linearly separable matrix, it is exceptionally well-suited for linear models, which can efficiently assign weights to individual sentiment-bearing words.

4.2.2 V2: TF-IDF Unigram + Bigram

The second version, V2, extends the TF-IDF vectorizer to include both unigrams and bigrams (pairs of consecutive words) [6]. The primary motivation for this expansion is to capture the local context that unigrams fundamentally ignore. Specifically, it addresses the prevalent negations identified during exploratory analysis; while V1 treats ”not” and ”good” as isolated features, V2 creates a distinct feature for ”not good”, allowing the model to accurately interpret sentiment inversion. The trade-off for this contextual enrichment is a massive expansion in feature dimensionality, which increases computational overhead and the potential for overfitting, necessitating careful hyperparameter tuning [7].

4.2.3 V3: Statistical Features

The third version, V3, pivots away from sparse vocabulary matrices entirely. Instead, it engineers dense, highly interpretable statistical features directly from the text. Conceptually, this includes attributes such as total character count, total word count, average word length, the ratio of uppercase letters, and explicit aggregate counts of predefined positive and negative sentiment dictionaries [8]. Because these features encapsulate the macro-structure and explicit sentiment density of a review into a small, dense vector, V3 is particularly well-suited for tree-based models and ensemble methods [9]. These algorithms excel at finding non-linear decision boundaries based on thresholding continuous numerical features, making V3 a highly complementary approach to the TF-IDF methods.

4.3 Model

To evaluate sentiment classification from different perspectives, this project uses five models: VADER, Logistic Regression, Linear Support Vector Classifier (LinearSVC), Decision Tree, and Random Forest. These models were chosen to cover three different ideas in sentiment analysis: a rule-based baseline, linear supervised classifiers for sparse text features, and non-linear tree-based methods for handcrafted statistical features. For experiments, the TF-IDF unigram+bigrams feature set is used with the linear classifiers, while the statistical feature set is used with the tree-based models.

4.3.1 VADER Baseline

VADER is used as a rule-based baseline [10]. Unlike machine learning models, it does not learn from labeled training data. Instead, it uses a predefined sentiment lexicon together with sentiment rules to assign a polarity score to each review. For every input review, the `SentimentIntensityAnalyzer` produces a compound score between -1 and 1. A review is then classified as positive if the compound score is greater than or equal to 0.05, and negative otherwise [10].

The main purpose of using VADER is to provide a simple baseline for comparison. It is easy to apply and does not require feature training or parameter tuning. It can also handle some useful sentiment patterns such as negation, degree words, and punctuation emphasis [10]. However, because it depends on a fixed dictionary and general rules, it may not match the language style of movie reviews very well. This makes it useful as a reference point, but usually weaker than supervised classifiers trained on the dataset itself.

4.3.2 Logistic Regression

Logistic Regression is one of the main supervised learning methods used in this project. It is a linear classifier that estimates the probability that a review belongs to the positive class. This model is widely used in text classification because it works well with high-dimensional sparse representations such as TF-IDF [11].

Given an input feature vector x, Logistic Regression computes

$$P(y = 1 \mid x) = \sigma(w^T x + b)$$

where $\sigma(z)$ is the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In this formula, x is the feature vector of a review, w is the set of learned weights, and b is the bias term. The quantity $w^T x + b$ can be understood as a sentiment score formed by adding together the feature values with different weights. Words or features that are more strongly related to positive sentiment will tend to push this score upward, while features related to negative sentiment will push it downward. Since this score can range from very large negative values to very large positive values, the sigmoid function is used to map it into the range from 0 to 1, so that it can be interpreted as a probability [11].

If the resulting probability is greater than or equal to 0.5, the model predicts the review as positive. Otherwise, it predicts the review as negative. Therefore, Logistic Regression can be viewed as a model that first computes a linear sentiment score and then converts that score into a probability for classification.

In this project, Logistic Regression is trained on the TF-IDF unigram+bigrams features. Grid search with 3-fold cross-validation is used to select the hyperparameters. The regularization strength is tested over several values of C, and different solvers are also compared. The tuning metric is the F1-score, since it balances precision and recall and is suitable for binary sentiment classification. After the best setting is selected, the final model is evaluated on the test set [11].

The strength of Logistic Regression is that it is simple, efficient, and highly effective for sparse text data. It is also relatively interpretable, because the learned weights show which words are strongly associated with positive or negative sentiment. Its main limitation is that it is still a linear model, so it may not fully capture more complex semantic relationships.

4.3.3 Linear Support Vector Classifier

LinearSVC is another important supervised model used in this project. Like Logistic Regression, it is also a linear classifier, but its learning objective is different. Instead of estimating class probabilities

directly, it tries to find a decision boundary that separates the two classes with the largest possible margin [12].

Its decision function can be written as

$$f(x) = w^T x + b$$

where x is the input feature vector, w is the learned weight vector, and b is the bias term. The value of $f(x)$ measures how strongly the review is pushed toward one class or the other. If $f(x)$ is positive, the review is predicted as belonging to the positive side of the boundary; if it is negative, it is predicted as belonging to the negative side. Therefore, the predicted label depends on which side of the separating hyperplane the input falls. The optimization objective of LinearSVC is based on hinge loss:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{n} \max(0, 1 - y_i(w^T x_i + b))$$

The first term, $\frac{1}{2}\|w\|^2$, tries to keep the weight vector small, which corresponds to making the margin as large as possible. A larger margin usually leads to better generalization on unseen data. The second term is the hinge loss, which penalizes points that are either classified incorrectly or classified correctly but too close to the decision boundary. In other words, the model does not only care about whether a point is on the correct side, but also whether it is far enough from the boundary. For each training example, if $y_i(w^T x_i + b) \geq 1$, the hinge loss is 0, which means the example is correctly classified and lies outside the margin. If this value is less than 1, then a penalty is added. The smaller the value is, the larger the penalty becomes. This means misclassified points and points inside the margin both affect the objective function [12].

The parameter C controls the trade-off between these two goals. A larger C puts more emphasis on reducing classification errors, so the model tries harder to classify all training examples correctly. A smaller C puts more emphasis on keeping a large margin, even if that allows some training errors. Therefore, C balances margin maximization and error minimization [12].

In this project, LinearSVC is also trained on the TF-IDF unigram+bigrams features. Grid search with 3-fold cross-validation is carried out over different values of C, and the F1-score is again used as the selection criterion. The final trained model is then tested on the held-out test set [11].

LinearSVC is often a very strong method for text classification because sparse text features are usually well suited to linear margin-based classifiers. It is efficient, robust, and often performs competitively on sentiment analysis tasks. However, like Logistic Regression, it still assumes a linear decision boundary and may miss more complicated patterns in the data.

4.3.4 Decision Tree

To compare the linear text models with a non-linear approach, a Decision Tree classifier is also included. Instead of using TF-IDF features, this model uses the statistical feature set extracted from the reviews. These features include information such as character count, word count, average word length, exclamation count, question count, negation count, positive word count, and negative word count.

A Decision Tree works by recursively splitting the feature space into smaller regions according to

feature thresholds. The goal of each split is to make the data in each branch more pure, meaning that the samples in a node become more dominated by a single class. In sentiment classification, this means a good split will separate reviews so that one side contains mostly positive reviews and the other side contains mostly negative reviews. In this project, class purity is measured using the Gini impurity criterion [13]. For a node with class proportions $p_1, p_2, \ldots, p_k$, the Gini impurity is defined as

$$G = 1 - \sum_{i=1}^{k} p_i^2$$

A lower Gini value means the node is more pure. If all samples in a node belong to one class, then the impurity becomes 0. During training, the tree chooses splits that reduce impurity as much as possible, so the resulting child nodes become more homogeneous than the parent node. To reduce overfitting, constraints are placed on the depth of the tree and the minimum number of samples allowed in each leaf node [13].

The main advantage of Decision Trees is that they are easy to understand. Their predictions come from a series of explicit rules, so it is possible to see which feature patterns lead to positive or negative decisions. However, a single tree can be unstable and sensitive to the training data. It may also be less effective than linear TF-IDF models when the most important sentiment signals come from sparse lexical patterns rather than simple numerical summaries.

4.3.5 Random Forest

Random Forest is used as an ensemble version of the Decision Tree model. Instead of building only one tree, it trains many trees on different bootstrap samples of the training data and combines their predictions. This usually improves generalization and makes the model less prone to overfitting than a single decision tree [14].

In this project, Random Forest is trained on the same statistical feature set as the Decision Tree. Each tree learns slightly different decision patterns from the data, and the final prediction is obtained by aggregating the outputs of all trees. Since each individual tree is still built by selecting splits that improve node purity, the forest as a whole can capture non-linear relationships while being more stable than one single tree. This allows the model to learn more reliable patterns from the handcrafted features and reduces the risk of making decisions based on only a few noisy splits [14].

The main strength of Random Forest is that it is generally more robust and accurate than a single Decision Tree. It can model complex interactions between features while reducing variance through ensembling. However, compared with linear models trained on TF-IDF features, it is less directly interpretable and may still be limited by the expressiveness of the statistical features themselves.

4.3.6 Model Selection

These five models provide a meaningful comparison across different sentiment analysis strategies. VADER gives a simple rule-based reference point without training [10]. Logistic Regression and LinearSVC test how well linear classifiers work with sparse lexical features [11, 12]. Decision Tree and Random Forest examine whether sentiment can also be captured through more compact statistical descriptions of the text [13, 14]. Together, these models make it possible to compare

different feature representations and learning mechanisms in a consistent way.

5 Experimental Setup

The experimental framework is designed to rigorously evaluate the efficacy of the proposed feature sets. The strictly isolated test set of 25,000 reviews will be held out entirely during the training phase and used solely for final model evaluation.

To provide a holistic view of model performance, we will report a suite of standard classification metrics based on the confusion matrix components: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). The primary metrics will include:

- **Accuracy:** The proportion of correctly classified reviews out of the total test set.
- **Precision:** The ratio of true positive predictions to the total predicted positives, indicating the model's exactness and its ability to avoid false alarms.
- **Recall:** The ratio of true positive predictions to the total actual positives, measuring the model's sensitivity in identifying sentiment.
- **F1-score:** The harmonic mean of Precision and Recall, providing a single, balanced metric that is particularly useful for summarizing the model's overall diagnostic ability.

6 Results and Discussion

6.1 Overall Model Comparison

Table 2 summarizes the test-set performance of the candidate models. Among all models, **LinearSVC** achieved the best overall result, with an accuracy of **0.8953** and an F1-score of **0.8961**. Logistic Regression performed almost as well, which suggests that the sentiment signal in this dataset is largely captured by linear decision boundaries over TF-IDF features. In contrast, the VADER lexicon baseline and the tree-based models were clearly weaker.

Table 2: Test-set performance comparison of the candidate models

Model	**Accuracy**	**Precision**	**Recall**	**F1**
LinearSVC	0.8953	0.8919	0.9004	0.8961
Logistic Regression	0.8928	0.8884	0.8993	0.8938
Random Forest	0.7461	0.7340	0.7746	0.7537
Decision Tree	0.7255	0.7100	0.7653	0.7366
VADER	0.7004	0.6532	0.8581	0.7418

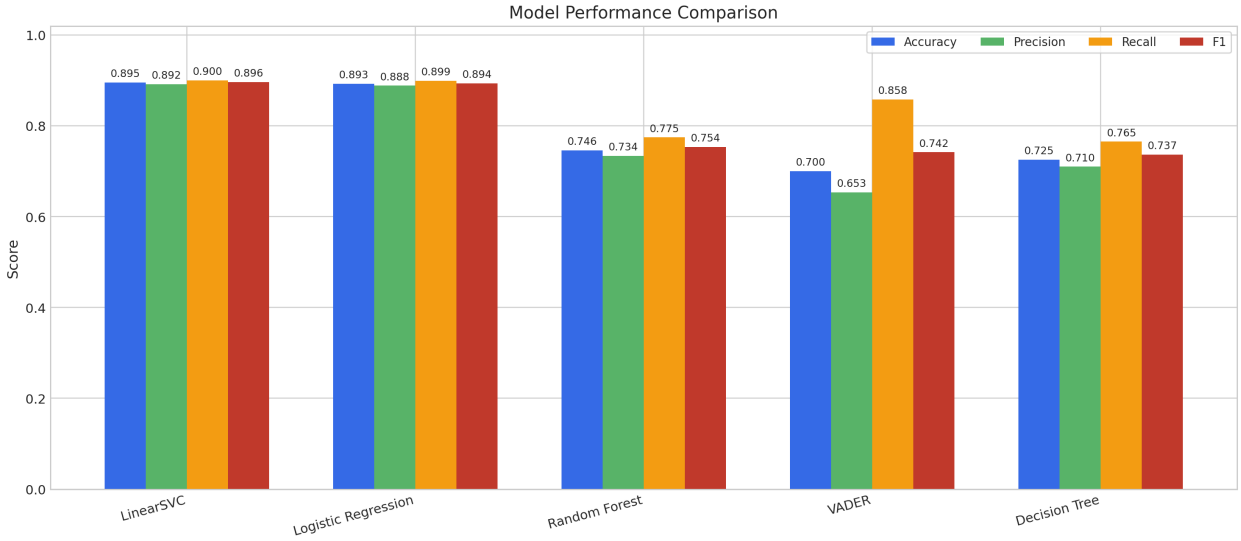

Figure 6: Comparison of the evaluated models on accuracy, precision, recall, and F1-score.

The result in Figure 6 confirms that the linear text models are the strongest choices for this task. VADER achieves relatively high recall, but its precision is much lower because the lexicon-based scoring is too coarse for nuanced movie reviews. The tree-based models benefit from interpretable numeric features, but they still lag behind the TF-IDF-based classifiers because the sentiment signal is expressed more strongly in word and phrase usage than in simple counts alone.

6.2 Error Analysis

To better understand the remaining mistakes, we inspected the misclassified test samples from the best-performing model. The most common issue is that many reviews are close to the decision boundary, so a small lexical shift can change the predicted label. A second major source of error is negation scope. Expressions such as “not good” or “not bad” are still difficult for bag-of-words style models because the local context is only partially captured. Long reviews are also harder to classify because they often mix plot summary, praise, criticism, and side comments in the same text.

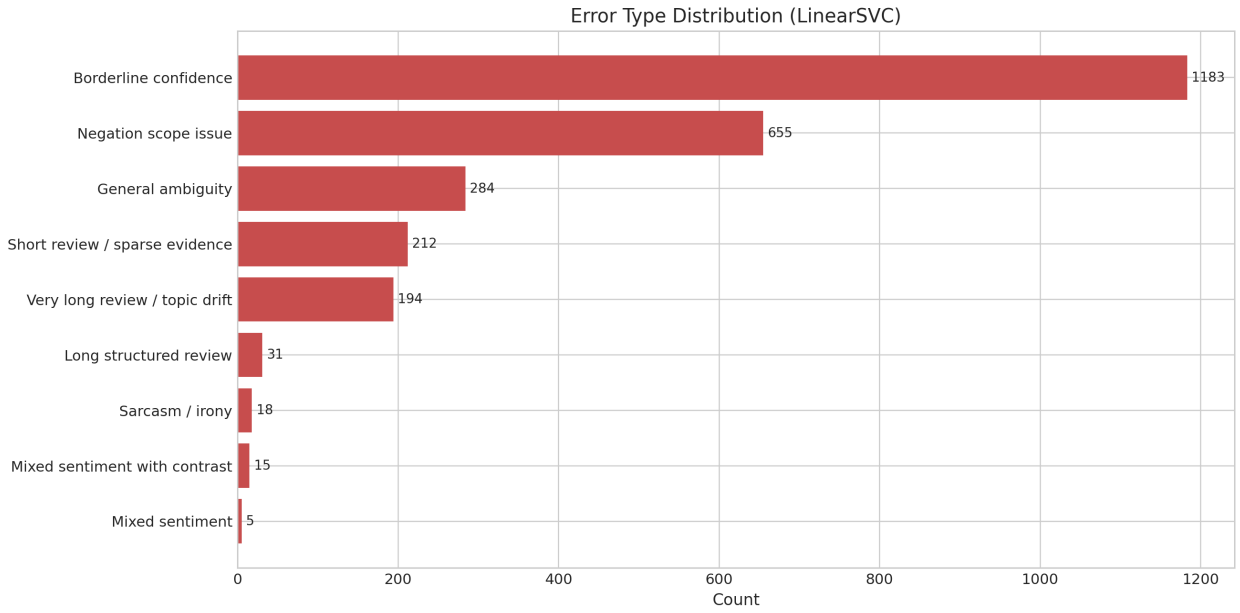

Figure 7: Distribution of the main error types observed in the misclassified reviews.

As shown in Figure 7, the largest group of mistakes comes from borderline-confidence cases, followed by reviews where negation is not handled cleanly. Short reviews are another noticeable source of error because they provide too little evidence for a stable decision. A smaller number of cases involve sarcasm, irony, or highly mixed sentiment. These examples suggest that the current feature design is strong for direct opinion words, but weaker for discourse-level cues such as contrast, irony, and sentiment reversal.

6.3 Feature Interpretation

The linear models provide the clearest interpretation of what drives the predictions. Positive words and phrases with strong weights include *excel*, *great*, *perfect*, *enjoy*, *love*, and *the best*. On the negative side, the strongest features include *aw*, *worst*, *bore*, *bad*, *wast*, and *disappoint*. These features are consistent with normal sentiment intuition, which gives some confidence that the model is learning meaningful patterns rather than relying on random correlations.

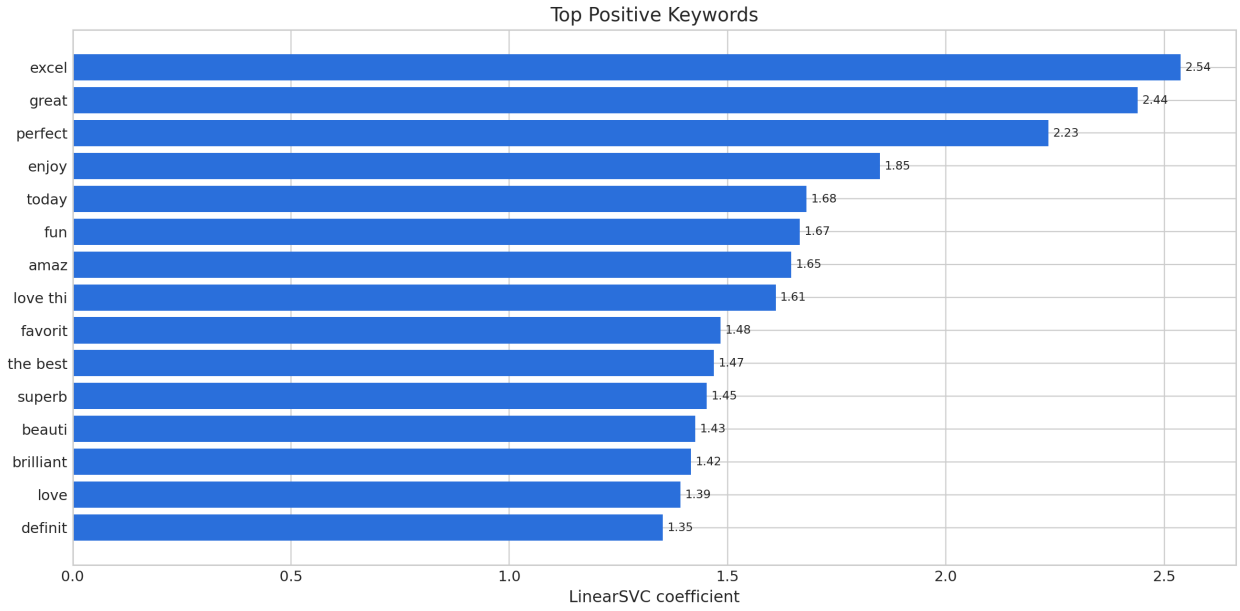

Figure 8: Top positive keywords learned by the LinearSVC model.

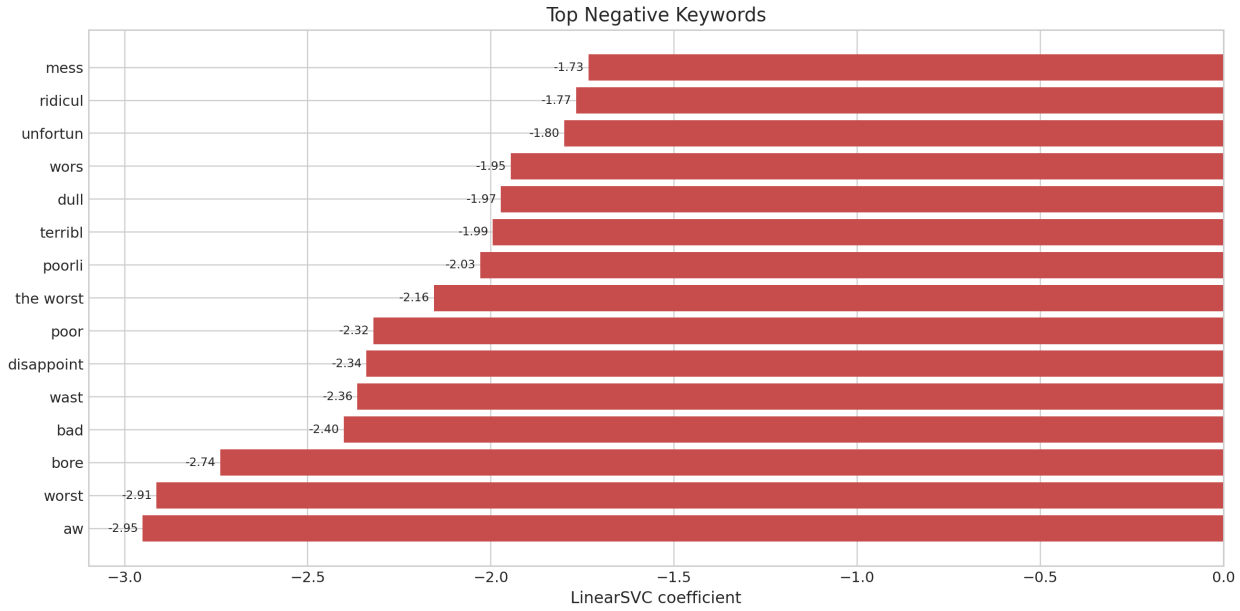

Figure 9: Top negative keywords learned by the LinearSVC model.

The keyword plots in Figures 8 and 9 show that the model is mainly driven by clear sentiment-bearing stems. This also reflects the preprocessing step, where stemming merges variants of the same word and makes the vocabulary more compact. For the tree-based models, the most important statistical features are `negative_word_count` and `positive_word_count`, which again confirms that simple sentiment density is highly informative. At the same time, the relatively lower importance of

punctuation-based features suggests that emotive markers such as exclamation points are useful only as secondary cues.

6.4 Influence of Review Length on Prediction

Review length has a noticeable but not perfectly monotonic influence on model performance. According to the distribution analysis, the cleaned reviews have a mean length of about 229 words and a median of 171 words, with a long right tail. Short reviews are often easy to classify when they contain an obvious opinion word, but they can also be unstable because a single token may dominate the prediction. Medium-length reviews, especially those between roughly 150 and 500 words, are somewhat harder because they tend to contain both plot description and opinion language.

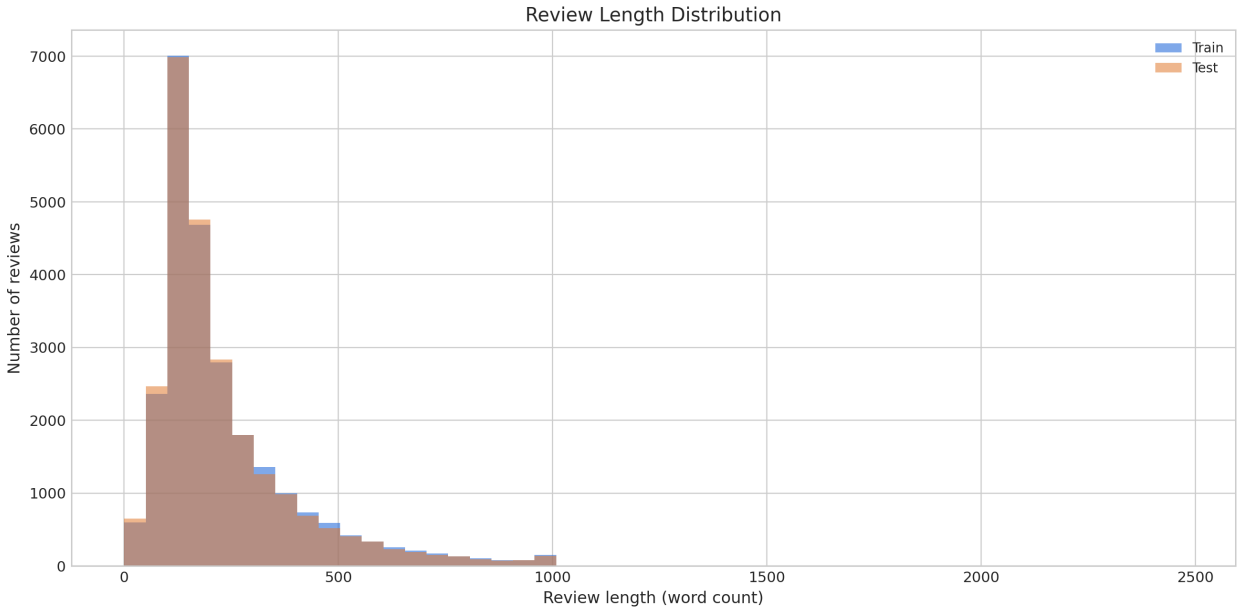

Figure 10: Distribution of review lengths in the cleaned dataset.

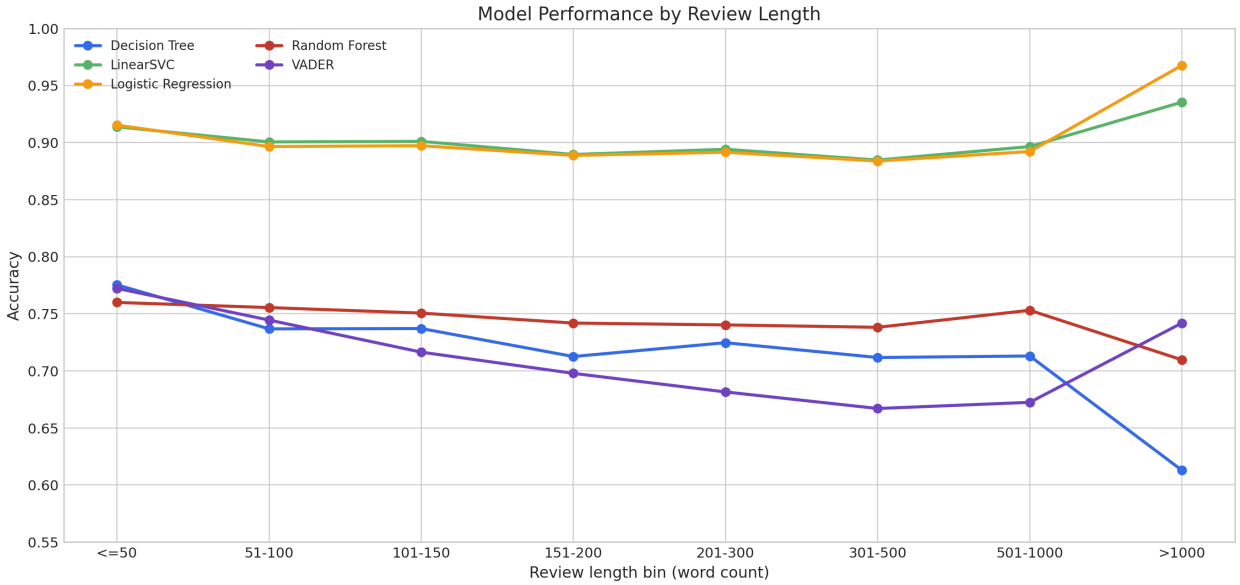

Figure 11: LinearSVC accuracy across different review-length bins.

Figure 11 shows that the model stays fairly stable across most length bins, but performance drops slightly for medium and long reviews. Very long reviews are rare, so the estimate for that group is less stable, but the general pattern still supports the qualitative observation from the error analysis: when a review contains multiple opinion shifts, it becomes harder for a bag-of-words classifier to decide which sentiment should dominate.

6.5 Summary of Analytical Findings

Overall, the results show that the sentiment task is best handled by linear models built on TF-IDF features. Logistic Regression and LinearSVC both work well, while the simpler lexicon baseline and tree-based classifiers are less effective. The main error sources are negation, mixed sentiment, sarcasm, and borderline cases near the decision boundary. In other words, the current pipeline is strong at capturing explicit opinion words, but it still struggles with context-sensitive language use.

From a practical perspective, the project suggests three takeaways. First, careful preprocessing matters because HTML artifacts and inconsistent formatting would otherwise pollute the vocabulary. Second, bigram or phrase-level features are important because they help with expressions such as "not good". Third, future improvements should focus on context-aware representations, for example adding stronger phrase modeling or using modern language models that can better capture sentiment shifts within a sentence.

7 Conclusion and Future Work

This project developed a complete sentiment analysis pipeline for IMDB movie reviews, including data cleaning, feature engineering, model training, and result analysis. Five models were compared: VADER, Logistic Regression, LinearSVC, Decision Tree, and Random Forest. The results show

that TF-IDF features with linear classifiers are the most effective approach for this task. Among all models, LinearSVC achieved the best overall performance, while Logistic Regression gave very similar results. This suggests that sentiment in the IMDB dataset can be captured well by sparse lexical features and linear decision boundaries. In contrast, VADER and the tree-based models performed noticeably worse, showing that simple lexicon rules or handcrafted statistical features are less effective for this dataset.

The project also shows that preprocessing and feature design are very important. Removing HTML tags, repeated punctuation, duplicates, and other noise helped create a cleaner feature space. Preserving negation was also necessary because words such as "not" can completely change sentiment meaning. In addition, adding bigrams improved the representation of short contextual phrases such as 'not good', which helped the linear models perform better.

At the same time, the error analysis shows that the current pipeline still has limitations. The most difficult cases include borderline reviews, sarcasm, negation scope, and long reviews with mixed opinions. These errors suggest that although traditional bag-of-words style models are strong and efficient, they still struggle with deeper semantic meaning and context-dependent language.

For future work, one possible improvement is to use more context-aware text representations, such as word embeddings or transformer-based models. These methods may help the system better understand longer reviews and more subtle sentiment changes. Another direction is to improve how the model handles difficult language patterns, especially negation, sarcasm, and reviews that contain both positive and negative opinions. This could be done by adding stronger phrase-level or sentence-level analysis. Overall, this project shows that traditional machine learning methods still work very well as strong baselines for sentiment analysis, but it also suggests that models with better contextual understanding could further improve the results.

References

[1] IMDB Movie Reviews Dataset. (2026). *IMDB Movie Reviews for Sentiment Analysis*. Available online: https://challengedata.ens.fr/participants/challenges/61/.

[2] Palomino, M.; Aider, F. (2022). *Evaluating the effectiveness of text pre-processing in sentiment analysis*. Appl Sci, 12(17), 8765. doi:10.3390/app12178765. Available online: https://www.mdpi.com/1806390?utm_source=chatgpt.com.

[3] HaCohen-Kerner, Y.; Miller, D.; Yigal, Y. (2020). *The influence of preprocessing on text classification using a bag-of-words representation*. PLoS ONE, 15(5), e0232525. doi:10.1371/journal.pone.0232525. Available online: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0232525&utm_source=chatgpt.com.

[4] Das, M.; Selvakumar, K.; Alphonse, P. J. A. (2023). *A Comparative Study on TF-IDF Feature Weighting Method and Its Analysis Using Unstructured Dataset*. arXiv preprint. Available online: https://arxiv.org/abs/2308.04037.

[5] Tambe, S.; Joshi, R.; Gupta, A.; Kanvinde, N.; Chitre, V. (2022). *Effects of Parametric and Non-Parametric Methods on High Dimensional Sparse Matrix Representations*. arXiv preprint. Available online: https://arxiv.org/abs/2202.02894.

[6] Bahrainian, S. A.; Dengel, A. (2015). *Sentiment analysis of texts by capturing underlying sentiment patterns*. Journal of Web Engineering. Available online: https://journals.sagepub.com/doi/10.3233/WEB-150309. Accessed on April 5, 2026.

[7] Wikimedia Foundation. (2026). *Document-term matrix*. Available online: https://en.wikipedia.org/wiki/Document-term_matrix. Accessed on April 5, 2026.

[8] Li, Q.; Mei, L.; Liu, H. (2022). *A Survey on Text Classification: From Traditional to Deep Learning Methods*. ACM Computing Surveys. Available online: https://dl.acm.org/doi/10.1145/3495162.

[9] Dogra, V.; Gupta, P.; Sharma, R. (2022). *A Complete Process of Text Classification System Using Machine Learning Techniques*. Journal of Clinical and Experimental Machine Learning. Available online: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9203176/.

[10] Hutto, C. J.; Gilbert, E. (2014). *VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text*. Proceedings of the International AAAI Conference on Web and Social Media, 8(1), 216–225. Available online: https://ojs.aaai.org/index.php/ICWSM/article/view/14550. Accessed on April 5, 2026.

[11] Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; Duchesnay, E. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. Available online: https://jmlr.org/papers/v12/pedregosa11a.html. Accessed on April 5, 2026.

[12] Cortes, C.; Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20, 273–297. Available online: https://link.springer.com/article/10.1007/BF00994018. Accessed on April 5, 2026.

[13] Breiman, L.; Friedman, J. H.; Olshen, R. A.; Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group. Available online: https://www.taylorfrancis.com/books/mono/10.1201/9781315139470/classification-regression-trees-leo-breiman-jerome-friedman-olshen-charles-stone. Accessed on April 5, 2026.

[14] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32. Available online: https://link.springer.com/article/10.1023/A:1010933404324. Accessed on April 5, 2026.

Appendix A – Statement on the Use of Large Language Models

A.1 Purpose of Use

The use of the large language model (LLM) in this report was strictly limited to the following purposes:

- Editing and proofreading: Checking grammar, improving sentence clarity, and refining expressions;
- Formatting and structural suggestions: Assisting in standardizing section titles, citation styles, and document layout;
- Non-substantive feedback: Suggesting removal of redundant wording and improving paragraph transitions.

A.2 Scope and Limitations

The LLM was not involved in generating any original academic or technical content. All research design, methodology, data analysis, algorithm implementation, and experimental framework were completed independently by the author.

A.3 Tool Information

Tool used: ChatGPT (OpenAI)
Date(s) of use: 20 November 2025